Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Мікросервісна архітектура та розподілені системи»

Практична робота №3
«Реалізація обов’язкових мікросервісів застосунку TripPack»

Виконала:
ст. гр. ІТШІ-23-3
Павленко М.Ю.

Прийняв:
викладач
Бібічков І.Є.

Харків 2026

3.1. Мета роботи

Метою практичної роботи є реалізація обов’язкових сервісів мікросервісного застосунку TripPack: сервісу автентифікації та авторизації, сервісу управління списками речей і сервісу категоризації речей. Додатково збережено Trip Service як окремий доменний сервіс для керування поїздками.

3.2. Вихідні дані та вимоги

Відповідно до ПЗ1, TripPack призначений для створення поїздок, формування списків речей, групування предметів за категоріями та контролю пакування. У ПЗ3 реалізовано обов’язкові функції: реєстрацію та вхід користувача, профіль; створення й редагування списків для поїздок; категоризацію та фільтрацію речей. Додатково реалізовано ролі user/admin, спільний доступ, класифікацію за назвою та автоматичне додавання рекомендованих речей за типом поїздки.

3.3. Архітектура реалізованого рішення

Система складається з Frontend, API Gateway, Auth Service, Trip Service, Packing Service та Categorization Service. Frontend надсилає HTTP-запити до Gateway. Gateway маршрутизує їх за URL-префіксами. Auth Service видає JWT; Trip Service керує поїздками; Packing Service зберігає списки та предмети; Categorization Service надає категорії, правила класифікації та шаблонні рекомендації для типів поїздок.

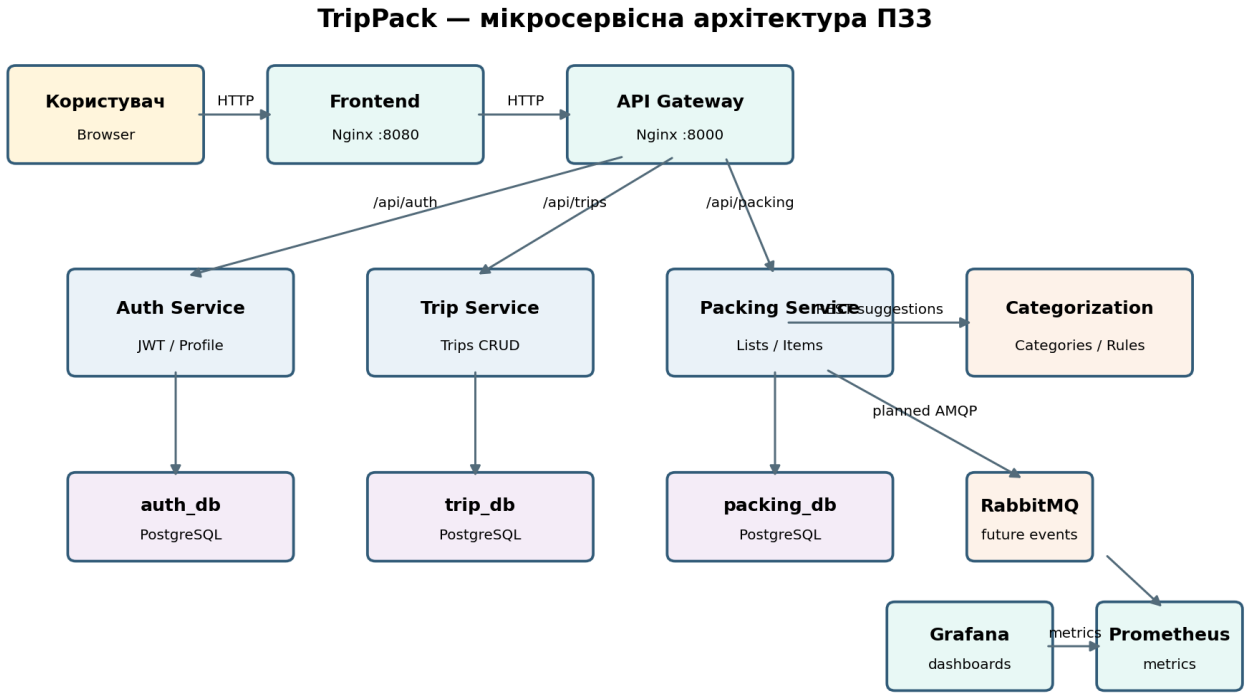


Рисунок 1 — Схема реалізованої мікросервісної архітектури TripPack

3.4. Реалізація Auth Service

Auth Service реалізує POST /auth/register, POST /auth/login, POST /auth/logout, GET/PUT /users/me. Паролі зберігаються у вигляді bcrypt-хешів. Після успішного входу повертається JWT з ідентифікатором користувача, роллю та строком дії 8 годин. Передбачені ролі user та admin; адміністративні операції перегляду користувачів і зміни ролі захищені перевіркою ролі. Logout у stateless JWT-моделі означає видалення токена на клієнті; для production потрібен механізм відкликання токенів або короткий строк дії.

3.5. Реалізація Trip Service

Trip Service реалізує створення, перегляд, редагування та видалення поїздок. Поїздка містить назву, тип, дату початку, дату завершення та опис. Сервіс перевіряє коректність дат і власника

запису: користувач бачить власні поїздки, адміністратор може переглядати записи всіх користувачів.

3.6. Реалізація Packing Service

Packing Service створює списки речей, пов’язує їх з trip_id, додає, редагує та видаляє предмети. Предмет має назву, категорію, кількість, примітку та прапорець is_packed. Реалізовано фільтрацію за категорією та статусом пакування. Власник може надати іншому користувачу спільний доступ до перегляду списку; редагування й видалення доступні власнику та адміністратору.

3.7. Реалізація Categorization Service

Categorization Service надає довідник категорій: одяг, техніка, документи, гігієна, аптечка та інше. Endpoint /classify виконує просту класифікацію назви предмета за ключовими словами. Endpoint /suggestions/{trip_type} повертає початковий набір речей для типів «відпочинок», «ділова» та «похід». Під час створення списку Packing Service звертається до цього сервісу та автоматично додає рекомендовані предмети.

3.8. Модель даних та ізоляція

Використано PostgreSQL з окремими базами auth_db, trip_db та packing_db. Auth Service володіє таблицею users; Trip Service — trips; Packing Service — packing_lists та items. Categorization Service у цій навчальній версії використовує статичний довідник і правила у коді, тому окрема БД для нього не створюється. Міжсервісні посилання зберігаються як ідентифікатори, без зовнішніх ключів між базами.

3.9. API та сценарій роботи

Основний сценарій: користувач реєструється, входить і отримує JWT; створює поїздку; створює список для trip_id і типу поїздки; Packing Service отримує рекомендації від Categorization Service; користувач переглядає список, фільтрує його за категорією та позначає предмети спакованими. OpenAPI/Swagger документація доступна окремо для кожного сервісу.

3.10. Контейнеризація та запуск

Для кожного сервісу створено Dockerfile на базі python:3.12-slim. Docker Compose описує Frontend, Gateway, чотири сервіси, PostgreSQL, RabbitMQ, Prometheus та Grafana. PostgreSQL ініціалізує три окремі бази; сервіси використовують змінні середовища та спільний JWT_SECRET. Для запуску необхідно скопіювати .env.example у .env, замінити секрети та виконати docker compose up --build -d. Перевірка стану: docker compose ps; журнали: docker compose logs -f.

3.11. Перевірка реалізації

Перевірка виконується через Swagger UI: реєстрація та вхід в Auth Service; отримання профілю; створення поїздки в Trip Service; створення списку в Packing Service; отримання автоматично доданих речей; фільтрація /lists/{id}/items?category=одяг; оновлення предмета з is_packed=true. Негативні сценарії: повторна реєстрація email (409), неправильний пароль (401), відсутній або недійсний JWT (401), спроба доступу до чужої поїздки/списку (404/403).

3.12. Межі реалізації

У ПЗ3 реалізовано синхронну REST-взаємодію. RabbitMQ збережено в Docker-інфраструктурі ПЗ2, але publishers/consumers та асинхронні бізнес-події не реалізовані, оскільки за планом курсу взаємозв’язок через брокер винесений у ПЗ4. Спільний доступ реалізовано для перегляду списку; спільне редагування та шаблони списків залишаються необов’язковими функціями наступних етапів.

3.13. Основні API-маршрути

Сервіс	Метод	Маршрут	Призначення
Auth	POST	/auth/register	Реєстрація
Auth	POST	/auth/login	Вхід і JWT
Auth	GET/PUT	/users/me	Профіль
Trip	POST/GET	/trips	Створення/перелік
Trip	GET/PUT/DELETE	/trips/{id}	Операції з поїздкою
Packing	POST/GET	/lists	Списки
Packing	POST	/lists/{id}/items	Додавання речі
Packing	GET	/lists/{id}/items	Фільтрація
Packing	PUT/DELETE	/items/{id}	Редагування/видалення
Categorization	GET	/categories	Категорії
Categorization	GET	/suggestions/{trip_type}	Рекомендації
Categorization	GET	/classify?name=...	Класифікація

Висновок

Під час практичної роботи №3 реалізовано основні функціональні компоненти TripPack відповідно до обов’язкових пунктів a–c. Створено Auth Service з реєстрацією, входом, профілем, JWT та ролями; Trip Service для керування поїздками; Packing Service для створення списків, додавання й редагування предметів, фільтрації та контролю пакування; Categorization Service для довідника категорій, класифікації та автоматичного формування початкового набору речей за типом поїздки. Збережено ізоляцію даних між сервісами через окремі бази PostgreSQL, а взаємодію організовано через API Gateway та REST API. Підготовлено Dockerfile-и й оновлений Docker Compose для запуску системи в єдиному середовищі. Отримана реалізація покриває основний сценарій користування TripPack і створює основу для наступного етапу — асинхронної взаємодії через RabbitMQ, розширення спільної роботи та тестування.